

Admin Dashboard Implementation Walkthrough

I have successfully built out the Monetchat Admin Dashboard, transitioning from a placeholder area to a fully functional marketplace management suite.

Security & Foundation

Admin Role: Added the `admin` role to the `UserRole` enum in

`schema.prisma`.

Access Control: Updated

`middleware.ts` to strictly enforce the `admin` role for all routes under `/admin`.

Authentication: Reused the existing JWT-based authentication system to ensure consistency and security.

Backend Infrastructure

Created a dedicated Admin API suite:

`GET /api/admin/stats`: Real-time KPI aggregation (User, Seller, Listing counts).

`GET /api/admin/users`: Paginated and searchable user management.

`GET /api/admin/products`: Listing moderation pipeline (List, Search, Approve/Reject).

`GET /api/admin/analytics`: Exposure of `SearchLog` and `ContactLog` data.

Dashboard Interface

Implemented a modern, dashboard-style UI using standard components:

Admin Sidebar: Persistent navigation for quick access to all management sections.

Stats Overview: High-level KPI cards for immediate situational awareness.

User Management: Searchable table displaying user roles, status, and join dates.

Listing Moderation: Interface to review products and toggle their visibility via Approve/Reject actions.

Search Insights: Visibility into real user queries and search performance metrics.

Local Testing Instructions

Promote a User: Manually set `role = 'admin'` for your user in the PostgreSQL database.

Access Dashboard: Navigate to `/admin`.

Manage Users: Use the search bar to find specific accounts.

Moderate Listings: Go to Listings and try rejecting/approving a product. Verify the status changes in the DB.

Remaining Gaps

Moderation Persistence: While we can reject listings, we don't yet have a separate `ModerationFlag` table for granular AI audit trails (only basic `rejectionReason`).

Suspension Logic: The `isActive` flag can be toggled via DB, but UI-driven user suspension (disabling login) is pending a dedicated API endpoint.